Université des Sciences et de la Technologie MB-Oran
Faculté des mathématiques et informatique
Département d'informatique
Niveau : Master 1

Polycopié de Cours — Bases de Données Avancées

CHAPITRE 1

Introduction aux bases de données

Chapitre 1 — Introduction aux bases de données

Ce premier chapitre pose les bases indispensables à la compréhension du reste du cours : il rappelle les fondements du modèle relationnel, situe les différentes familles de bases de données les unes par rapport aux autres, décrit l'architecture interne d'un SGBD et détaille les mécanismes transactionnels qui garantissent la fiabilité des données. Une bonne maîtrise de ces notions est indispensable pour aborder sereinement la programmation SQL avancée, le modèle objet-relationnel et les architectures NoSQL et distribuées des chapitres suivants.

1.1 Rappels sur les concepts fondamentaux

1.1.1 Bases de données et SGBD : définitions et historique

Une base de données est une collection organisée, structurée et persistante de données, partagée entre plusieurs utilisateurs ou applications, décrivant les activités d'une ou plusieurs organisations liées entre elles. Elle se distingue d'un simple ensemble de fichiers par l'existence d'un schéma explicite décrivant la structure des données et par l'indépendance qu'elle offre entre cette structure et les programmes qui l'exploitent.

Un Système de Gestion de Bases de Données (SGBD) est le logiciel qui permet de créer, alimenter, interroger, mettre à jour et administrer une base de données. Historiquement, les premiers systèmes des années 1960 reposaient sur le modèle hiérarchique (IMS d'IBM), dans lequel les données sont organisées en arborescence, puis sur le modèle réseau (CODASYL), qui autorise des relations plus complexes entre enregistrements sous forme de graphe. Ces deux modèles imposaient de connaître les chemins d'accès physiques aux données, rendant les programmes très dépendants de l'organisation physique du stockage.

En 1970, Edgar F. Codd (chercheur chez IBM) propose dans son article fondateur « A Relational Model of Data for Large Shared Data Banks » le modèle relationnel, fondé sur la théorie mathématique des ensembles et des relations. Ce modèle introduit une indépendance forte entre la structure logique des données (les tables) et leur implémentation physique, ce qui simplifie considérablement l'écriture des programmes et a permis l'essor des langages déclaratifs comme SQL, normalisé pour la première fois en 1986 (SQL-86) puis régulièrement enrichi (SQL-92, SQL:1999, SQL:2003, SQL:2011, SQL:2016, SQL:2023).

Un SGBD assure un ensemble de fonctions essentielles qu'il convient de bien distinguer :

- Indépendance physique et logique : les applications ne dépendent pas de l'organisation physique des fichiers.
- Contrôle de la redondance : la duplication non maîtrisée des données est limitée grâce à la normalisation (voir 1.1.4).
- Partage des données : plusieurs utilisateurs et applications accèdent simultanément aux mêmes données de façon cohérente.
- Application des contraintes d'intégrité : le SGBD garantit automatiquement le respect de règles métier (unicité, référence, domaine).
- Sécurité et contrôle des accès : chaque utilisateur ne peut effectuer que les opérations pour lesquelles il est habilité (voir chapitre 7).
- Gestion de la concurrence d'accès et reprise après panne : propriétés détaillées en section 1.4.

1.1.2 Le modèle relationnel : formalisation

Le modèle relationnel repose sur la notion mathématique de relation. Formellement, une relation R est un sous-ensemble du produit cartésien de domaines D1 × D2 × ... × Dn. Concrètement, une relation se représente sous forme d'une table à deux dimensions :

- Un attribut est une colonne de la table, associée à un domaine de valeurs (par exemple : entier, chaîne de caractères, date).
- Un tuple (ou n-uplet) est une ligne de la table, c'est-à-dire une occurrence de la relation.
- Le schéma d'une relation est constitué de son nom et de la liste de ses attributs typés, par exemple : Employe(id_emp, nom, prenom, salaire, id_dept).
- Le degré d'une relation est son nombre d'attributs ; sa cardinalité est son nombre de tuples à un instant donné.

Plusieurs types de contraintes garantissent la cohérence des données stockées :

- Clé candidate : ensemble minimal d'attributs déterminant de façon unique chaque tuple d'une relation.
- Clé primaire : clé candidate choisie pour identifier de façon unique et non nulle chaque tuple (contrainte PRIMARY KEY).
- Clé étrangère : attribut (ou ensemble d'attributs) d'une relation référençant la clé primaire d'une autre relation (ou de la même relation), garantissant l'intégrité référentielle (contrainte FOREIGN KEY).
- Contrainte de domaine : restriction des valeurs autorisées pour un attribut (type, plage, expression CHECK).
- Contrainte de non-nullité (NOT NULL) : interdiction de la valeur indéterminée NULL pour un attribut donné.

> ***Exemple de schéma :*** *Departement(id_dept, nom_dept) ; Employe(id_emp, nom, prenom, salaire, id_dept REFERENCES Departement). L'attribut id_dept d'Employe est une clé étrangère vers Departement : tout employé doit être rattaché à un département existant.*

1.1.3 L'algèbre relationnelle

L'algèbre relationnelle constitue le fondement théorique du langage SQL : c'est un ensemble d'opérateurs qui, appliqués à des relations, produisent de nouvelles relations. Chaque requête SQL peut en principe être traduite en une expression d'algèbre relationnelle, ce qui permet à l'optimiseur du SGBD de raisonner sur des transformations équivalentes pour choisir le plan d'exécution le plus efficace (voir chapitre 7).

Opérateur	Notation	Effet	Équivalent SQL
Sélection	σcondition(R)	Filtre les tuples de R vérifiant la condition	WHERE
Projection	πattributs(R)	Ne conserve que certains attributs (colonnes)	SELECT
Union	R ∪ S	Tuples présents dans R ou dans S	UNION
Différence	R − S	Tuples de R absents de S	EXCEPT / MINUS
Intersection	R ∩ S	Tuples communs à R et S	INTERSECT
Produit cartésien	R × S	Toutes les combinaisons de tuples de R et S	CROSS JOIN
Jointure naturelle	R ⋈ S	Produit cartésien filtré sur les attributs communs	JOIN ... ON
Division	R ÷ S	Tuples de R associés à tous les tuples de S	NOT EXISTS imbriqué

1.1.4 La normalisation des schémas relationnels

La normalisation est un processus de conception qui vise à organiser les attributs d'une base de données en relations afin de minimiser la redondance et d'éviter les anomalies de mise à jour, d'insertion et de suppression. Elle s'appuie sur la notion de dépendance fonctionnelle : un attribut B dépend fonctionnellement d'un attribut A (noté A → B) si, à chaque valeur de A, correspond une unique valeur de B.

- Première forme normale (1NF) : chaque attribut contient une valeur atomique (indivisible) ; pas de groupes répétitifs ni d'attributs multivalués.
- Deuxième forme normale (2NF) : la relation est en 1NF et chaque attribut non-clé dépend de la totalité de la clé primaire (pas de dépendance partielle vis-à-vis d'une clé composite).
- Troisième forme normale (3NF) : la relation est en 2NF et aucun attribut non-clé ne dépend transitivement de la clé primaire (c'est-à-dire via un autre attribut non-clé).
- Forme normale de Boyce-Codd (BCNF) : version renforcée de la 3NF où toute dépendance fonctionnelle non triviale a pour source une clé candidate.

> ***Exemple d'anomalie évitée par la normalisation :*** *Dans une table unique Commande(id_commande, id_client, nom_client, adresse_client, produit, prix), le nom et l'adresse du client sont répétés à chaque commande (redondance), et la suppression de la dernière commande d'un client entraîne la perte de ses coordonnées (anomalie de suppression). La décomposition en deux tables Client(id_client, nom_client, adresse_client) et Commande(id_commande, id_client, produit, prix) élimine ces anomalies.*

1.1.5 Les langages du SGBD

Un SGBD relationnel expose plusieurs sous-langages, chacun ayant une responsabilité précise dans le cycle de vie de la base de données :

Langage	Sigle	Rôle	Instructions principales
Langage de Définition de Données	LDD (DDL)	Créer et modifier les structures (schémas)	CREATE, ALTER, DROP, TRUNCATE
Langage de Manipulation de Données	LMD (DML)	Interroger et modifier le contenu des tables	SELECT, INSERT, UPDATE, DELETE
Langage de Contrôle de Données	LCD (DCL)	Gérer les droits d'accès des utilisateurs	GRANT, REVOKE
Langage de Contrôle de Transaction	LCT (TCL)	Valider ou annuler les transactions	COMMIT, ROLLBACK, SAVEPOINT

SQL demeure le langage de référence pour l'ensemble de ces fonctions, normalisé par l'ISO/IEC, bien que chaque éditeur propose des extensions procédurales propriétaires pour dépasser les limites du SQL purement déclaratif : PL/SQL pour Oracle, Transact-SQL (T-SQL) pour Microsoft SQL Server, PL/pgSQL pour PostgreSQL. Ces extensions seront étudiées en détail au chapitre 2.

1.1.6 Le langage SQL : vue d'ensemble

SQL (Structured Query Language) est le langage déclaratif de référence pour interroger un SGBD relationnel : il constitue la partie la plus utilisée du LMD présenté ci-dessus. Contrairement à un langage procédural, où le programmeur décrit pas à pas la suite d'opérations à exécuter, une requête SQL décrit uniquement le résultat souhaité (le QUOI) ; c'est l'optimiseur de requêtes qui détermine ensuite le plan d'exécution le plus efficace (le COMMENT) pour l'obtenir.

Une requête de consultation s'appuie sur plusieurs clauses, chacune correspondant en principe à un opérateur de l'algèbre relationnelle (voir 1.1.3) :

Clause	Rôle	Obligatoire ?
SELECT	Projection : liste des colonnes (ou expressions) à retourner	Oui
FROM	Table(s) source(s), éventuellement combinées par une jointure	Oui
WHERE	Sélection : filtre appliqué ligne par ligne, avant tout regroupement	Non
GROUP BY	Regroupe les tuples partageant les mêmes valeurs, en vue d'agrégats (COUNT, SUM, AVG, MIN, MAX)	Non
HAVING	Filtre appliqué APRÈS le regroupement, sur les agrégats eux-mêmes	Non
ORDER BY	Trie le résultat final selon une ou plusieurs colonnes (ASC/DESC)	Non

Reprenons le schéma Departement(id_dept, nom_dept) / Employe(id_emp, nom, prenom, salaire, id_dept) introduit en 1.1.2. La requête suivante affiche, pour chaque département, le salaire moyen des employés dont le salaire dépasse 30 000, en ne conservant que les départements dont ce salaire moyen excède 40 000, triés du plus élevé au plus faible :

```
SELECT d.nom_dept, AVG(e.salaire) AS salaire_moyen
FROM Employe e
JOIN Departement d ON e.id_dept = d.id_dept
WHERE e.salaire > 30000
GROUP BY d.nom_dept
HAVING AVG(e.salaire) > 40000
ORDER BY salaire_moyen DESC;
```

L'ordre syntaxique dans lequel les clauses sont écrites (SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY) ne correspond pas à leur ordre logique d'exécution par le SGBD, qui est en réalité FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Cet ordre logique explique par exemple

pourquoi WHERE ne peut pas filtrer sur un agrégat (celui-ci n'existe pas encore à ce stade, d'où la nécessité de HAVING) et pourquoi un alias défini dans le SELECT (comme salaire_moyen ci-dessus) reste malgré tout utilisable dans la clause ORDER BY, exécutée après.

1.2 Types de bases de données

Le choix d'un type de base de données dépend de la structure des données à stocker, du volume attendu, de la vitesse de lecture et d'écriture requise et des contraintes de cohérence imposées par le domaine métier. Il n'existe pas de solution universelle : un système d'information moderne combine généralement plusieurs types de bases de données, une approche appelée polyglotte persistence, chaque composant applicatif étant adossé au type de stockage le plus adapté à son usage.

Type	Modèle de données	Exemples	Cas d'usage typiques
Relationnelles	Tables normalisées, schéma fixe	Oracle, PostgreSQL, MySQL, SQL Server	Gestion, ERP, transactions financières
NoSQL	Clé-valeur, document, colonne, graphe	Redis, MongoDB, Cassandra, Neo4j	Big Data, temps réel, schéma variable
Orientées objet	Objets, classes, héritage	db4o, ObjectDB, Versant	Applications orientées objet complexes (CAO, jeux vidéo)
Objet-relationnel	Tables enrichies de types complexes	Oracle, PostgreSQL, IBM DB2	Données structurées mais complexes (géospatial, multimédia)
Entrepôts de données (Data Warehouse)	Schémas en étoile / flocon, orienté colonnes	Amazon Redshift, Snowflake, Teradata	Analyse décisionnelle (OLAP), reporting
Bases XML natives	Arbres XML / semi-structuré	BaseX, MarkLogic	Documents structurés, échanges normalisés (SOAP, factures)

1.3 Architecture d'un SGBD : client-serveur, architectures distribuées

1.3.1 Architecture interne d'un SGBD

Indépendamment de son architecture de déploiement, un SGBD s'organise en interne selon plusieurs couches logicielles coopérantes, du traitement de la requête déclarative jusqu'à l'accès au support physique de stockage :

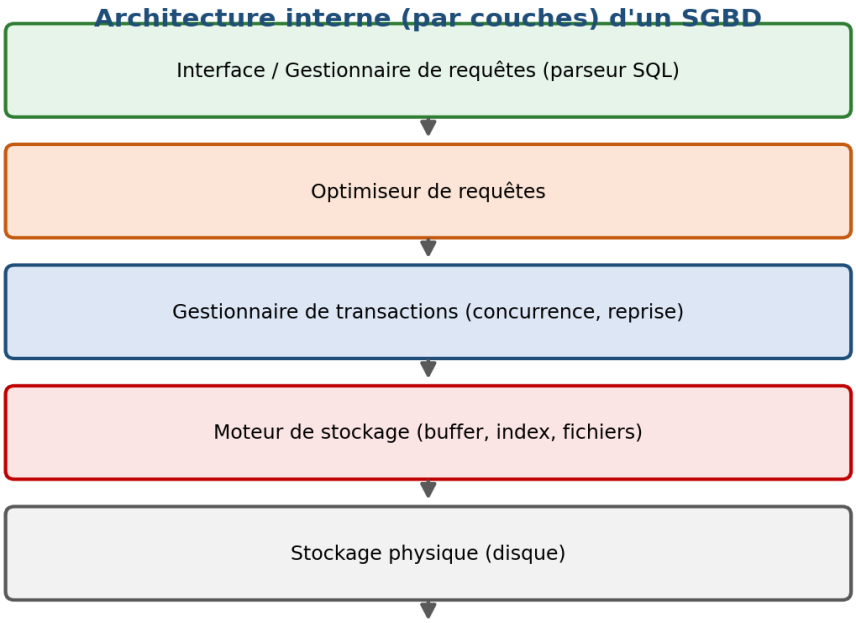


Figure 1.1 — Architecture interne en couches d'un SGBD

- Le gestionnaire de requêtes analyse syntaxiquement (parsing) la requête SQL soumise et vérifie sa validité sémantique (existence des tables, des colonnes, des droits).
- L'optimiseur de requêtes transforme la requête déclarative en un ou plusieurs plans d'exécution physiques candidats, puis sélectionne le moins coûteux à l'aide de statistiques (voir chapitre 7).
- Le gestionnaire de transactions coordonne l'exécution concurrente des transactions et assure les propriétés ACID (voir section 1.4).
- Le moteur de stockage gère les structures physiques : gestion du cache mémoire (buffer pool), organisation des pages de données, index, journalisation.
- Le stockage physique correspond enfin aux fichiers effectivement écrits sur le disque ou tout autre support persistant.

1.3.2 Architecture client-serveur

Dans une architecture client-serveur à deux niveaux (two-tier), le poste client (application ou outil de requêtage) envoie directement des requêtes SQL au serveur de base de données, qui les exécute et retourne les résultats. Ce modèle, simple à mettre en œuvre, montre ses limites en termes de sécurité (le client dispose d'un accès direct à la base) et de montée en charge (chaque client ouvre sa propre connexion, ce qui limite le nombre d'utilisateurs simultanés).

L'architecture à trois niveaux (three-tier) introduit une couche applicative intermédiaire, le serveur d'application, qui encapsule la logique métier, valide les données avant de les transmettre, et mutualise les connexions à la base de données grâce à des pools de connexions. Cette architecture est aujourd'hui la plus répandue pour les applications web.

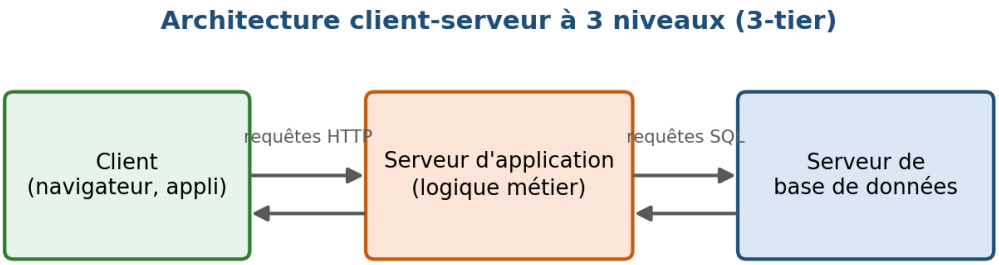


Figure 1.2 — Architecture client-serveur à trois niveaux

1.3.3 Vers les architectures distribuées

Une base de données distribuée répartit les données sur plusieurs sites (nœuds) reliés par un réseau, tout en offrant à l'utilisateur une vue logique unique de l'ensemble : c'est le principe de transparence de distribution, qui masque à l'utilisateur la localisation physique réelle des données. Ce principe de transparence se décline en plusieurs sous-principes : transparence de localisation (l'utilisateur ignore où se trouvent physiquement les données), transparence de fragmentation (il ignore comment une relation est découpée entre les sites) et transparence de réplication (il ignore combien de copies existent et laquelle est consultée).

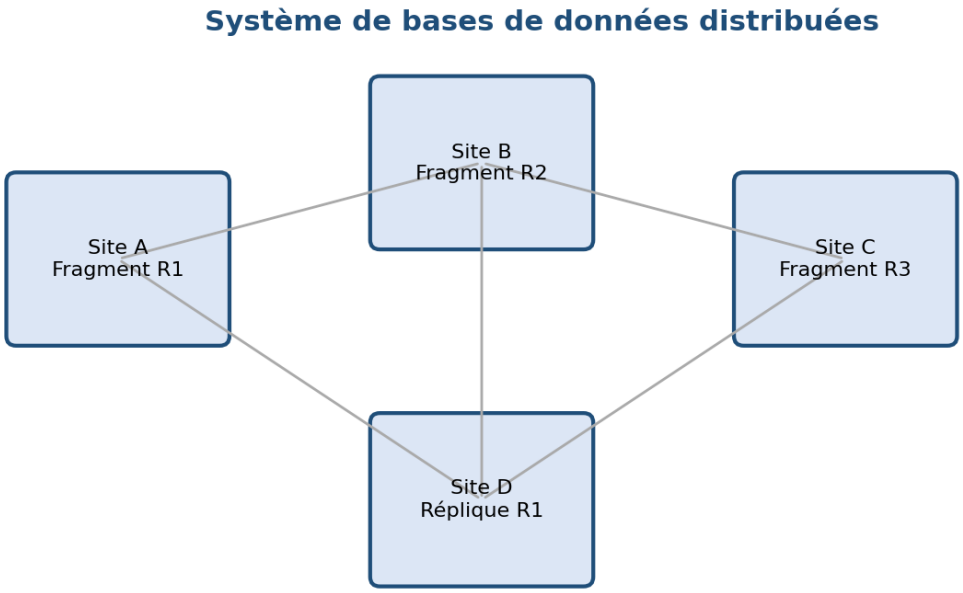


Figure 1.3 — Vue logique d'un système de bases de données distribuées

On distingue les architectures homogènes, où tous les sites exécutent le même SGBD et partagent un schéma global conçu de manière centralisée, des architectures hétérogènes ou fédérées, qui intègrent des SGBD différents (parfois de fournisseurs différents) au moyen d'une couche de médiation (middleware) chargée de traduire les requêtes et les formats de données entre systèmes. Les concepts de fragmentation, réplication et allocation seront développés en détail au chapitre 5, consacré spécifiquement aux bases de données distribuées.

1.4 Concepts de transaction, concurrence et récupération de données

1.4.1 La transaction et son cycle de vie

Une transaction est une unité logique de traitement regroupant une ou plusieurs opérations de lecture ou d'écriture qui doivent être exécutées de manière indivisible du point de vue du reste du système. Au cours de son exécution, une transaction traverse plusieurs états :

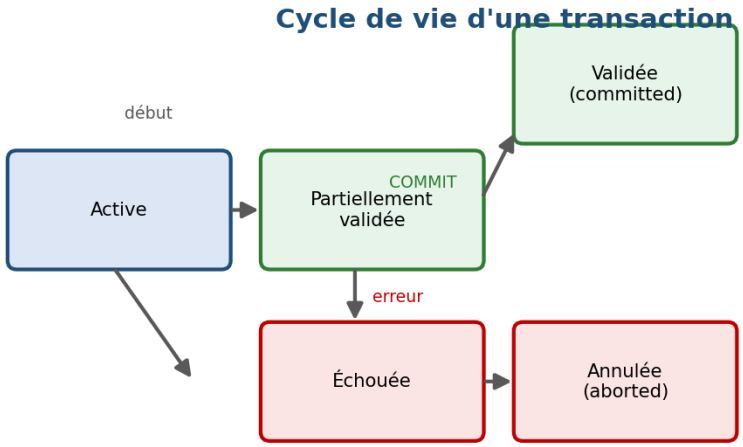


Figure 1.4 — Cycle de vie d'une transaction

- Active : la transaction est en cours d'exécution, ses opérations sont en train de s'exécuter.
- Partiellement validée : la dernière instruction vient de s'exécuter, mais la validation définitive (écriture durable) n'a pas encore eu lieu.
- Validée (committed) : la transaction s'est terminée avec succès, ses effets sont rendus permanents et visibles des autres transactions.
- Échouée : une erreur est survenue (violation de contrainte, panne) empêchant la poursuite normale de la transaction.
- Annulée (aborted) : la transaction a été annulée (ROLLBACK), et la base est ramenée à l'état qui précédait son démarrage.

Exemple concret d'une transaction : virement bancaire

L'exemple le plus classique pour comprendre concrètement ce qu'est une transaction est le virement bancaire, qui illustre pourquoi plusieurs opérations doivent impérativement être regroupées en une seule unité indivisible. Débiter un compte et créditer un autre compte sont deux opérations SQL distinctes ; pourtant, du point de vue métier, il est inconcevable que l'une s'exécute sans l'autre — une panne survenant juste après le débit et avant le crédit ferait littéralement disparaître de l'argent du système.

```
BEGIN TRANSACTION;

  -- Opération 1 : débiter le compte source de 500
  UPDATE Compte SET solde = solde - 500 WHERE id_compte = 101;

  -- Opération 2 : créditer le compte destinataire de 500
  UPDATE Compte SET solde = solde + 500 WHERE id_compte = 202;

  -- Vérification métier : le compte source ne doit jamais être négatif
  -- (dans un vrai système, cette vérification précéderait le débit)

COMMIT;   -- Valide les DEUX opérations ensemble, de façon définitive
-- Si une erreur survient avant COMMIT (ex. compte 202 inexistant),
-- ROLLBACK annule le débit déjà effectué : aucune trace de l'opération
-- partielle ne subsiste, comme si la transaction n'avait jamais eu lieu.
```

Cet exemple illustre directement l'atomicité (les deux UPDATE forment un tout indivisible), et prépare la compréhension des trois autres propriétés ACID détaillées ci-après : la cohérence (le total des soldes de tous les comptes du système reste inchangé avant et après le virement), l'isolation (un autre utilisateur consultant le solde du compte 101 pendant l'exécution de la transaction ne doit jamais voir un état intermédiaire où seul le débit a été appliqué) et la durabilité (une fois COMMIT exécuté, le virement reste acquis même si le serveur tombe en panne la seconde suivante).

1.4.2 Les propriétés ACID

Pour être qualifiée de transaction au sens strict, une unité de traitement doit respecter quatre propriétés fondamentales, connues sous l'acronyme ACID :

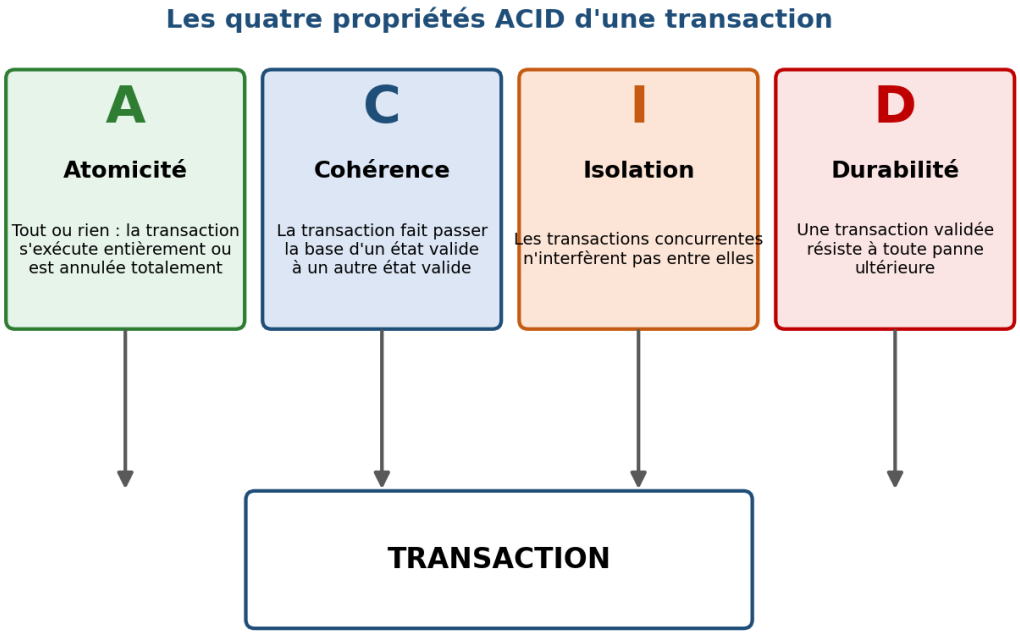


Figure 1.5 — Les quatre propriétés ACID d'une transaction

- Atomicité : la transaction s'exécute entièrement ou pas du tout. Si une opération échoue en cours de route, toutes les opérations déjà effectuées doivent être annulées (principe du « tout ou rien »).
- Cohérence : la transaction fait passer la base de données d'un état cohérent (respectant toutes les contraintes d'intégrité) à un autre état également cohérent, y compris en cas d'échec (annulation totale).
- Isolation : les effets intermédiaires d'une transaction en cours ne doivent pas être visibles par les autres transactions tant qu'elle n'a pas été validée, comme si les transactions s'exécutaient en série.
- Durabilité : une fois qu'une transaction a été validée (COMMIT), ses effets doivent survivre à toute panne système ultérieure (coupure électrique, crash du serveur).

1.4.3 Les anomalies de concurrence

L'exécution simultanée de plusieurs transactions, nécessaire pour offrir de bonnes performances à de nombreux utilisateurs, peut provoquer des anomalies si elle n'est pas correctement contrôlée. Chacune est illustrée ci-dessous par un scénario chronologique concret impliquant deux transactions T1 et T2 s'exécutant en parallèle.

a) Lecture sale (dirty read)

Une transaction T1 lit une donnée modifiée par une transaction T2 non encore validée ; si T2 est finalement annulée, T1 a travaillé sur une donnée qui n'a jamais réellement existé du point de vue de la base.

```
Temps  | T1 (lecture)                  | T2 (écriture)
-------|-------------------------------|-----------------------------
  t1   |                               | UPDATE Compte SET solde=1500
       |                               | WHERE id=101;  (non validé)
  t2   | SELECT solde FROM Compte      |
       | WHERE id=101;  → lit 1500     |
  t3   |                               | ROLLBACK;  (T2 annulée !)
  t4   | -- T1 a lu 1500, une valeur   |
       | -- qui n'a JAMAIS existé      |
       | -- réellement dans la base    |
```

b) Lecture non répétable (non-repeatable read)

Une transaction relit deux fois la même donnée au cours de son exécution et obtient des valeurs différentes, car une autre transaction l'a modifiée et validée entre-temps.

```
Temps  | T1 (deux lectures)              | T2 (écriture)
-------|---------------------------------|-----------------------------
  t1   | SELECT solde FROM Compte        |
       | WHERE id=101;  → lit 1000       |
  t2   |                                 | UPDATE Compte SET solde=800
       |                                 | WHERE id=101; COMMIT;
  t3   | SELECT solde FROM Compte        |
       | WHERE id=101;  → lit 800  (!)   |
       | -- deux lectures, dans la MÊME  |
       | -- transaction, deux résultats  |
```

c) Lecture fantôme (phantom read)

Une transaction ré-exécute deux fois la même requête d'ensemble (par exemple un COUNT avec condition) et obtient un nombre de résultats différent, car une autre transaction a inséré ou supprimé des tuples correspondant à la condition entre-temps.

```
Temps  | T1 (deux comptages)                | T2 (insertion)
-------|------------------------------------|-----------------------
  t1   | SELECT COUNT(*) FROM Commande      |
       | WHERE montant > 1000;  → lit 12    |
  t2   |                                    | INSERT INTO Commande
       |                                    | (montant) VALUES (1500);
       |                                    | COMMIT;
  t3   | SELECT COUNT(*) FROM Commande      |
       | WHERE montant > 1000;  → lit 13(!) |
       | -- une ligne "fantôme" est apparue |
```

d) Perte de mise à jour (lost update)

Deux transactions lisent la même valeur, la modifient chacune de leur côté puis la valident ; la seconde validation écrase silencieusement la modification de la première, qui est perdue sans qu'aucune erreur ne soit signalée.

```
Temps  | T1 (+200 sur le solde)          | T2 (+100 sur le solde)
-------|---------------------------------|-----------------------------
  t1   | SELECT solde FROM Compte        |
       | WHERE id=101;  → lit 1000       |
  t2   |                                 | SELECT solde FROM Compte
       |                                 | WHERE id=101;  → lit 1000
  t3   | UPDATE Compte SET solde=1200    |  -- 1000 + 200, calculé en
       | WHERE id=101; COMMIT;           |  -- mémoire par T1
  t4   |                                 | UPDATE Compte SET solde=1100
       |                                 | WHERE id=101; COMMIT;
       |                                 | -- 1000 + 100, calculé par T2
       |                                 | -- AVANT de voir le COMMIT de T1
  t5   | -- Solde final = 1100 : le +200 de T1 a totalement disparu,   |
       | -- écrasé par le +100 de T2, sans le moindre message d'erreur |
```

1.4.4 Protocoles de contrôle de concurrence

Les SGBD mettent en œuvre des protocoles de contrôle de concurrence pour garantir la sérialisabilité des exécutions, c'est-à-dire garantir que le résultat d'une exécution concurrente est équivalent à celui d'une exécution séquentielle (en série) de ces mêmes transactions, dans un ordre quelconque — et donc pour empêcher précisément les quatre anomalies présentées ci-dessus.

Verrouillage à deux phases (2PL)

Le verrouillage (locking) consiste à associer à chaque donnée accédée un verrou partagé (lecture, plusieurs transactions peuvent le détenir simultanément) ou exclusif (écriture, une seule transaction à la fois). Le protocole à deux phases (Two-Phase Locking) impose que chaque transaction acquière tous ses verrous avant d'en libérer le moindre : on distingue ainsi une phase de croissance (acquisition des verrous) suivie d'une phase de décroissance (libération des verrous). Ce protocole garantit la sérialisabilité des exécutions, au prix d'un risque d'interblocage (deadlock) entre deux transactions qui attendent mutuellement la libération d'un verrou détenu par l'autre.

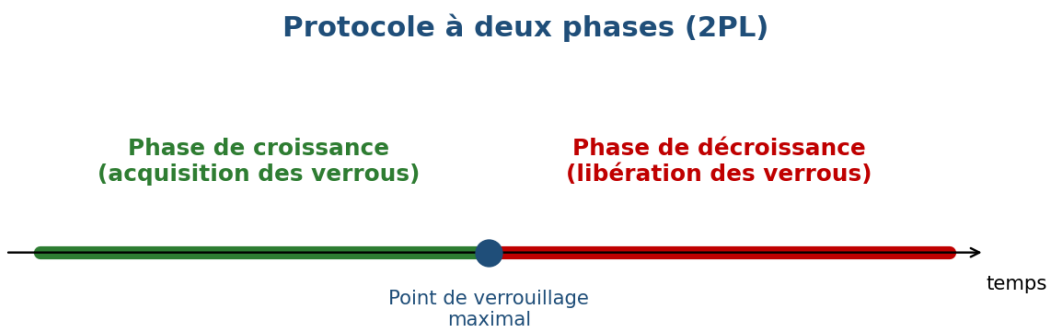


Figure 1.6 — Le protocole de verrouillage à deux phases (2PL)

Exemple : 2PL empêchant la perte de mise à jour

Reprenons l'exemple de perte de mise à jour ci-dessus, cette fois protégé par le verrouillage à deux phases : T2 doit attendre que T1 libère son verrou exclusif avant de pouvoir lire à son tour la donnée, ce qui empêche mécaniquement l'anomalie.

```
Temps  | T1                                  | T2
-------|-------------------------------------|-------------------------------
  t1   | Verrou EXCLUSIF sur Compte(101)     |
       | SELECT solde ... → lit 1000         |
  t2   |                                     | Demande verrou sur Compte(101)
       |                                     | → BLOQUÉE (verrou déjà pris)
  t3   | UPDATE solde=1200; COMMIT;          |
       | Libère le verrou                    |
  t4   |                                     | Obtient enfin le verrou
       |                                     | SELECT solde ... → lit 1200 (!)
  t5   |                                     | UPDATE solde=1300; COMMIT;
       | -- Résultat correct : 1000+200+100 = 1300, plus AUCUNE perte  |
```

Exemple d'interblocage (deadlock) sous 2PL

```
Temps  | T1                                  | T2
-------|-------------------------------------|-------------------------------
  t1   | Verrou EXCLUSIF sur Compte(101)     |
  t2   |                                     | Verrou EXCLUSIF sur Compte(202)
  t3   | Demande verrou sur Compte(202)      |
       | → BLOQUÉE (détenu par T2)           |
  t4   |                                     | Demande verrou sur Compte(101)
       |                                     | → BLOQUÉE (détenu par T1)
       | -- INTERBLOCAGE : chacune attend l'autre indéfiniment.        |
       | -- Le SGBD détecte le cycle d'attente et ANNULE l'une des     |
       | -- deux transactions (victime) pour débloquer la situation.  |
```

Estampillage temporel

Le protocole d'estampillage temporel (timestamp ordering) attribue à chaque transaction un horodatage unique au moment de son démarrage, qui détermine l'ordre de sérialisation souhaité. Chaque donnée conserve la trace du dernier horodatage de lecture et d'écriture qui l'a affectée ; toute tentative d'accès qui violerait l'ordre chronologique imposé par les horodatages entraîne l'annulation (et le redémarrage) de la transaction fautive. Ce protocole évite les interblocages mais peut provoquer davantage d'annulations de transactions en cas de forte contention.

```
T1 démarre avec horodatage TS(T1) = 100
T2 démarre avec horodatage TS(T2) = 105   (T2 est "plus jeune" que T1)
```

```
Règle : si une transaction plus JEUNE tente d'écrire une donnée déjà
        lue par une transaction plus ÂGÉE qui n'a pas encore validé,
        cela violerait l'ordre chronologique → REJET et redémarrage

  T2 tente d'écrire Compte(101), déjà lu par T1 (TS=100 < TS=105)
  → Écriture de T2 ACCEPTÉE (respecte l'ordre TS(T1) < TS(T2))

  Mais si T1 (plus ÂGÉE) tente ensuite de lire une donnée déjà écrite
  par T2 (plus JEUNE, TS=105 > TS=100) :
  → LECTURE DE T1 REJETÉE : T1 est redémarrée avec un nouvel horodatage
    (elle "arriverait trop tard" par rapport à l'ordre déjà établi)
```

Contrôle optimiste de concurrence

Le contrôle optimiste de concurrence part du principe que les conflits sont rares : les transactions s'exécutent librement sur une copie locale des données (phase de lecture), puis, juste avant la validation, une phase de validation vérifie qu'aucun conflit n'est apparu avec d'autres transactions validées entre-temps ; en l'absence de conflit, la phase d'écriture rend les modifications visibles. Cette approche, bien adaptée aux environnements à faible contention (peu de conflits), est largement utilisée par les bases de données NoSQL et les architectures distribuées modernes.

```
PHASE DE LECTURE (locale, sans verrou)
  T1 lit solde = 1000, calcule nouveau_solde = 1200 (en mémoire locale)

PHASE DE VALIDATION (juste avant COMMIT)
  Le SGBD vérifie : la donnée solde a-t-elle été modifiée par une
  AUTRE transaction validée depuis que T1 l'a lue ?
    SI NON : aucun conflit → passer à la phase d'écriture
    SI OUI : conflit détecté → ANNULER T1, qui devra recommencer
             depuis le début avec les données à jour

PHASE D'ÉCRITURE (si validation réussie)
  Appliquer nouveau_solde = 1200 de façon définitive, COMMIT
```

1.4.5 Niveaux d'isolation SQL

La norme SQL définit quatre niveaux d'isolation, du plus permissif au plus strict, permettant d'arbitrer explicitement entre cohérence stricte et performance (un niveau d'isolation strict multiplie les verrous posés et donc réduit le parallélisme effectif) :

- **READ UNCOMMITTED** : niveau le plus permissif ; une transaction peut lire des données modifiées mais non encore validées par une autre transaction (lecture sale). Rarement utilisé en pratique.
- **READ COMMITTED** : une transaction ne voit que des données validées (les lectures sales sont éliminées), mais deux lectures successives de la même donnée peuvent donner des résultats différents si une autre transaction l'a modifiée et validée entre-temps.
- **REPEATABLE READ** : garantit qu'une donnée relue plusieurs fois dans la même transaction reste identique, mais n'empêche pas l'apparition de nouvelles lignes (lectures fantômes) insérées par d'autres transactions concurrentes.
- **SERIALIZABLE** : niveau le plus strict ; les transactions concurrentes s'exécutent comme si elles étaient sérialisées les unes après les autres, éliminant toutes les anomalies (lecture sale, non répétable, fantôme), au prix d'un parallélisme fortement réduit.

Le tableau ci-dessous résume les anomalies encore possibles à chaque niveau :

Niveau d'isolation	Lecture sale	Lecture non répétable	Lecture fantôme
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Empêchée	Possible	Possible
REPEATABLE READ	Empêchée	Empêchée	Possible
SERIALIZABLE	Empêchée	Empêchée	Empêchée

Le niveau READ COMMITTED est le niveau par défaut de la plupart des SGBD relationnels (Oracle, PostgreSQL, SQL Server) car il offre un bon compromis pratique. Le niveau SERIALIZABLE, le plus strict, est réservé aux traitements pour lesquels la moindre incohérence est inacceptable (transferts bancaires par exemple), au prix d'une forte réduction du parallélisme.

1.4.6 Récupération après panne (recovery)

La reprise après panne repose principalement sur la journalisation anticipée des écritures (Write-Ahead Logging, WAL) : le principe fondamental est qu'aucune modification ne doit être écrite dans les fichiers de données avant que l'enregistrement correspondant n'ait été écrit, de façon durable, dans le journal des transactions (log).

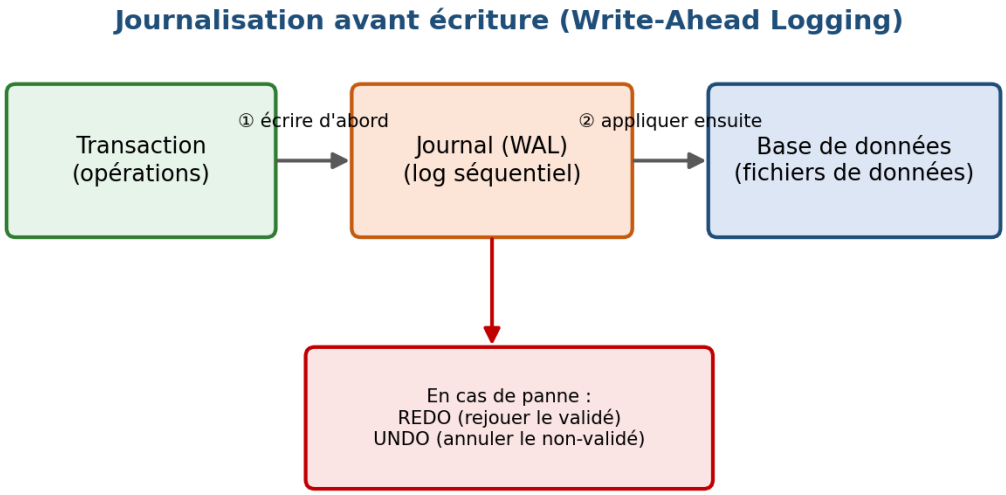


Figure 1.7 — Principe de la journalisation anticipée (Write-Ahead Logging)

En cas de panne, le système parcourt le journal pour restaurer la base dans un état cohérent. Le protocole ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), développé par IBM et largement adopté par les SGBD commerciaux (DB2, SQL Server, et dans une variante adaptée par PostgreSQL et MySQL/InnoDB), en constitue l'illustration la plus aboutie et la plus étudiée.

Les trois structures de données d'ARIES

Avant d'examiner l'algorithme lui-même, il faut bien comprendre trois structures sur lesquelles ARIES s'appuie constamment.

a) Le numéro de séquence de journal (LSN) et le PageLSN

Chaque opération enregistrée dans le journal reçoit un numéro appelé LSN (Log Sequence Number). Les LSN sont strictement croissants et permettent de connaître sans ambiguïté l'ordre chronologique des opérations.

```
LSN   | Opération
------|-----------------
100   | T1 modifie P1
110   | T2 modifie P2
120   | T1 modifie P1
130   | T1 COMMIT

L'opération LSN 130 est donc plus RÉCENTE que l'opération LSN 100.
```

Chaque page de la base mémorise également un PageLSN : le LSN de la dernière modification qui lui a été effectivement appliquée sur disque. En comparant le LSN d'une opération journalisée au PageLSN de la page concernée, ARIES sait immédiatement si cette opération a déjà été appliquée ou non.

```
Cas 1 :  Journal → LSN 120 modifie P1
         Disque  → P1.PageLSN = 120
         → la modification LSN 120 est DÉJÀ présente sur la page :
```

```
           inutile de la refaire

Cas 2 :  Journal → LSN 120 modifie P1
         Disque  → P1.PageLSN = 100
         → la modification LSN 120 n'a PAS encore été appliquée :
           elle devra être REFAITE
```

> ***À retenir :*** *Le LSN permet de connaître l'ordre chronologique des opérations ; le PageLSN permet de savoir, pour une page donnée, si une modification journalisée a déjà été appliquée ou si elle doit être rejouée.*

b) La table des transactions actives

Cette table indique quelles transactions étaient encore en cours d'exécution (non validées) au moment précis du crash.

```
Juste avant la panne :
  T1 → COMMIT
  T2 → encore en cours
  T3 → COMMIT
  T4 → encore en cours
          ↓
        CRASH

Table des transactions actives obtenue :
  Transaction | Dernier LSN
  T2          | 240
  T4          | 280

(T1 et T3 n'y figurent PAS : elles avaient déjà validé)
```

> ***À retenir :*** *La table des transactions actives répond à la question : « Quelles transactions n'étaient pas terminées au moment du crash ? » — ce sont précisément celles que la phase UNDO devra annuler.*

c) La table des pages sales (Dirty Page Table)

Une page est dite sale (dirty) lorsqu'elle a été modifiée en mémoire sans que cette modification ait nécessairement déjà été écrite, de façon durable, sur le disque.

```
Disque :   P1 → solde = 1000
               ↓  (T1 modifie P1)
Mémoire :  P1 → solde = 1500   ← P1 est maintenant SALE
                                 (la version mémoire diffère du disque)
```

La table des pages sales mémorise, pour chaque page concernée, son recLSN : le LSN de la toute première modification qui l'a rendue sale — c'est-à-dire le plus ancien point à partir duquel cette page pourrait ne pas être à jour sur disque.

```
Page | recLSN
-----|-------
P1   | 100
P5   | 150
P8   | 180

min(recLSN) = 100 → c'est LÀ qu'ARIES devra démarrer la phase REDO
```

> ***À retenir :*** *La table des pages sales indique quelles pages doivent être vérifiées lors du REDO, et surtout à partir de quel LSN il est nécessaire de commencer ce parcours du journal.*

Vue d'ensemble : trois phases, trois questions

Après un crash, ARIES répond successivement à trois questions simples, chacune correspondant à une phase de l'algorithme :

```
                CRASH
                  │
                  ▼
  ANALYSE  : « Que s'est-il passé ? »
                  │
                  ▼
  REDO     : « Qu'est-ce qu'il faut refaire ? »
                  │
                  ▼
  UNDO     : « Qu'est-ce qu'il faut annuler ? »
```

ANALYSE ne modifie encore rien : elle se contente de reconstituer, à partir du journal, la table des transactions actives et la table des pages sales décrites ci-dessus. REDO reconstruit ensuite l'état EXACT de la base au moment du crash, y compris le travail de transactions non validées. UNDO, enfin, annule ce travail non validé — et seulement celui-ci.

Algorithme ARIES en détail

L'algorithme se déroule en trois phases séquentielles, chacune parcourant le journal dans un sens précis.

```
PHASE 1 — ANALYSE (parcourt le journal du dernier CHECKPOINT jusqu'à la fin)
  1. Initialiser Table_Transactions et Table_Pages_Sales à partir
     du dernier enregistrement de checkpoint
  2. POUR chaque enregistrement du journal, dans l'ordre chronologique :
       SI c'est un début de transaction : l'ajouter à Table_Transactions
       SI c'est une modification de page :
         SI la page n'est pas déjà dans Table_Pages_Sales :
           l'ajouter avec recLSN = LSN de cet enregistrement
       SI c'est un COMMIT ou un ABORT :
         retirer la transaction de Table_Transactions
  3. À la fin : Table_Transactions = transactions ACTIVES au crash
                Table_Pages_Sales = pages à vérifier lors du REDO

PHASE 2 — REDO (parcourt le journal depuis le plus petit recLSN trouvé)
  1. Point de départ = min(recLSN) parmi toutes les pages de
     Table_Pages_Sales (inutile de remonter plus loin : ces pages
     étaient déjà proprement écrites sur disque avant ce point)
  2. POUR chaque enregistrement de modification, dans l'ordre :
       SI la page concernée n'est PAS dans Table_Pages_Sales,
          OU si le PageLSN de la page ≥ LSN de l'enregistrement :
         → NE RIEN FAIRE (déjà appliqué, réappliquer serait redondant)
       SINON :
         RÉAPPLIQUER l'opération (REDO), y COMPRIS pour les
         transactions non validées : on reconstitue D'ABORD l'état
         EXACT du crash, avant de décider quoi annuler

PHASE 3 — UNDO (parcourt le journal en sens INVERSE, du plus récent
                au plus ancien, uniquement pour les transactions
                encore actives dans Table_Transactions après l'Analyse)
  1. POUR chaque transaction active au crash (non validée) :
       Parcourir ses enregistrements du plus récent au plus ancien
       Pour chaque modification : appliquer l'AVANT-IMAGE (annuler)
       Écrire un enregistrement de compensation (CLR — Compensation
       Log Record) journalisant cette annulation elle-même,
       afin qu'une PANNE PENDANT LA REPRISE ne fasse pas tout
       recommencer depuis le début
  2. Une fois toutes les opérations annulées, marquer la
     transaction comme définitivement ABORTED
```

Le détail le plus subtil, et le plus souvent absent des présentations simplifiées de REDO/UNDO, est l'usage des enregistrements de compensation (CLR) : ARIES journalise même les opérations d'annulation elles-mêmes. Si une SECONDE panne survient en pleine phase de reprise (juste après l'annulation d'une opération, par exemple),

le système sait, grâce aux CLR déjà écrits, exactement quelles annulations ont déjà été effectuées — il n'a donc jamais besoin de recommencer l'intégralité du travail de reprise depuis le début.

Exemple complet : scénario de crash pas à pas

Considérons le scénario suivant, avec trois transactions et un point de reprise, pour illustrer concrètement le déroulement des trois phases d'ARIES.

```
LSN 100 : CHECKPOINT (aucune transaction active, aucune page sale)
LSN 110 : T1 démarre
LSN 120 : T1 écrit Compte(101).solde : 1000 → 800
LSN 130 : T2 démarre
LSN 140 : T2 écrit Compte(202).solde : 500 → 700
LSN 150 : T1 COMMIT
LSN 160 : T3 démarre
LSN 170 : T3 écrit Compte(303).solde : 2000 → 1500
LSN 180 : T2 écrit Compte(202).solde : 700 → 900
   —     : *** PANNE SYSTÈME ***
```

État réel des pages sur le disque au moment du crash (imprévisible à l'avance : il dépend uniquement de la date à laquelle le système d'exploitation avait déjà physiquement écrit chaque page) :

```
Page | Valeur sur disque au crash | Dernier PageLSN
-----|----------------------------|-----------------
P1   | 800                        | 120   (déjà à jour)
P2   | 500                        | < 140 (pas encore écrite)
P3   | 2000                       | < 170 (pas encore écrite)
```

– Déroulement de la phase d'Analyse : construction pas à pas de la table des transactions

```
Au CHECKPOINT (LSN 100)         : Table_Transactions = Ø

LSN 110 : T1 démarre             : Table_Transactions = { T1 }
LSN 120 : T1 modifie P1          : (T1 reste dans la table)
LSN 130 : T2 démarre             : Table_Transactions = { T1, T2 }
LSN 140 : T2 modifie P2          : (T2 reste dans la table)
LSN 150 : T1 COMMIT              : T1 RETIRÉE
                                  : Table_Transactions = { T2 }
LSN 160 : T3 démarre             : Table_Transactions = { T2, T3 }
LSN 170 : T3 modifie P3          : (T3 reste dans la table)
LSN 180 : T2 modifie P2          : (T2 reste dans la table)

TABLE FINALE au moment du crash :
  Transaction | Dernier LSN | État
  T2          | 180         | ACTIVE
  T3          | 170         | ACTIVE
  (T1 n'apparaît pas : elle avait déjà validé au LSN 150)
```

Construction pas à pas de la table des pages sales, au cours du même parcours

```
LSN 120 : T1 modifie P1  → P1 devient sale, recLSN(P1) = 120
LSN 140 : T2 modifie P2  → P2 devient sale, recLSN(P2) = 140
LSN 170 : T3 modifie P3  → P3 devient sale, recLSN(P3) = 170
LSN 180 : T2 modifie P2  → P2 est DÉJÀ dans la table :
                           son recLSN NE CHANGE PAS (reste 140)

TABLE FINALE :
  Page | recLSN
  P1   | 120
  P2   | 140
  P3   | 170

  → min(recLSN) = 120 : c'est ici que débutera la phase REDO
  → T2 et T3 devront être ANNULÉES (UNDO) ; T1 reste acquise (REDO seul)
```

– Déroulement de la phase REDO : comparaison systématique au PageLSN

ARIES parcourt le journal à partir du LSN 120 et compare, pour chaque opération, le LSN de l'enregistrement au PageLSN effectivement constaté sur la page concernée. Un point essentiel : durant cette phase, ARIES ne se demande PAS encore si la transaction sera finalement conservée ou annulée — il reconstruit d'abord, sans exception, l'état exact du crash.

```
LSN 120 (T1, P1 : 1000→800)
   Journal : modification LSN 120       Disque : P1.PageLSN = 120
   → 120 = PageLSN(P1)  ⇒  DÉJÀ APPLIQUÉE : RIEN À FAIRE

LSN 140 (T2, P2 : 500→700)
   Journal : modification LSN 140       Disque : P2 = 500 (ancienne)
   → PAS encore appliquée  ⇒  RÉAPPLIQUER : P2.solde = 700,
                                            PageLSN(P2) = 140

LSN 170 (T3, P3 : 2000→1500)
   Journal : modification LSN 170       Disque : P3 = 2000 (ancienne)
   → PAS encore appliquée  ⇒  RÉAPPLIQUER : P3.solde = 1500,
                                            PageLSN(P3) = 170

LSN 180 (T2, P2 : 700→900)
   → PAS encore appliquée  ⇒  RÉAPPLIQUER : P2.solde = 900,
                                            PageLSN(P2) = 180

ÉTAT APRÈS REDO (état EXACT du crash, T2 et T3 pourtant non validées) :
  Compte 101 (P1) = 800     Compte 202 (P2) = 900
  Compte 303 (P3) = 1500
```

– Déroulement de la phase UNDO : annulation en ordre inverse

ARIES consulte la table des transactions actives (T2, T3) et parcourt le journal en sens INVERSE, du plus récent au plus ancien, en annulant chaque opération concernée et en journalisant systématiquement un CLR pour chacune.

```
LSN 180 (T2, P2 : 700→900) : ANNULER → P2.solde revient à 700
                               (CLR écrit : "annulation du LSN 180 faite")
LSN 170 (T3, P3 : 2000→1500) : ANNULER → P3.solde revient à 2000
                               (CLR écrit)
LSN 140 (T2, P2 : 500→700) : ANNULER → P2.solde revient à 500
                               (CLR écrit)
→ T2 et T3 entièrement annulées, marquées ABORTED
```

Résultat final après reprise complète

Compte (page)	Valeur finale	Transaction	Décision
101 (P1)	800	T1	COMMIT → conservé
202 (P2)	500	T2	annulée → valeur d'origine restaurée
303 (P3)	2000	T3	annulée → valeur d'origine restaurée

Ce scénario illustre la logique parfois contre-intuitive d'ARIES, résumée par la formule : « ARIES fait d'abord REDO pour reconstruire l'état du crash, puis UNDO pour éliminer le travail des transactions incomplètes. » On REFAIT donc d'abord absolument tout ce qui est journalisé, y compris le travail de transactions qui seront ensuite annulées, afin de reconstituer avec certitude l'état exact du crash à partir d'un journal fiable — avant de DÉFAIRE proprement, et dans l'ordre inverse, le travail des seules transactions non validées.

Le rôle des points de reprise (checkpoints)

Des points de reprise sont établis périodiquement : ils enregistrent un instantané cohérent de l'état du système (transactions actives, pages sales), ce qui limite la portion du journal qu'il est nécessaire de parcourir lors d'une reprise, réduisant ainsi le temps d'indisponibilité du système après incident. Sans checkpoint, la phase d'Analyse

devrait remonter jusqu'au tout début du journal, potentiellement des jours ou des semaines d'historique ; avec des checkpoints réguliers (par exemple toutes les quelques minutes), elle ne remonte que jusqu'au dernier checkpoint précédant le crash.